CERA Electricity Tariff Comparison Tool — Azure Deployment Brief (Revised)

Prepared by: Regulatory Affairs / CERA

Date: July 2026

Purpose: Recommendation for hosting the CERA Tariff Comparison Tool on Microsoft Azure

Revision note: This version addresses questions raised on the June 2026 draft.

---

1. Background

The CERA Electricity Tariff Comparison Tool is a web application developed and currently running locally on a CERA workstation. It is ready for deployment. This document describes the recommended hosting approach — Microsoft Azure App Service — and the rationale for choosing it over an on-premises server deployment.

> 💬 **EP (IT):** Through MS 365?

The application uses CERA's existing Microsoft 365 / Azure tenant for sending email notifications (OTP login codes, tariff approval/rejection notices, supplier invitations). All email is sent via the Microsoft Graph API (Mail.Send) using an App Registration already created within CERA's tenant. No separate email provider is introduced — the tool operates entirely within the Microsoft ecosystem already in use at CERA.

> 📝 **PK:** It uses my personal CERA email for now with a Key — this needs to be switched to a shared/service mailbox before go-live.

A separate document covers the full technical overview of the application. This document focuses on Azure deployment architecture, security, and the actions required from IT.

---

2. Why Azure and not a CERA server

Both options are technically viable. Azure is recommended for the following reasons:

Security maintenance is handled by Microsoft, not CERA IT

Operating system security patches, platform vulnerability fixes, and infrastructure hardening are applied automatically by Microsoft. On Azure App Service, the underlying OS and Python

runtime are maintained by Microsoft — CERA IT does not need to manually apply OS patches or monitor for platform vulnerabilities.

> **EP (IT):** Who will update Python version, PostgreSQL etc?

Regarding Python and runtime version updates: Azure App Service allows the Python runtime version to be changed in a single configuration step in the Azure Portal (Settings → Configuration → General settings → Python version). Microsoft notifies customers when a runtime version approaches end-of-life. Updating is a one-click change and a redeploy — it does not require touching any application code. PostgreSQL (if adopted at full production) is similarly managed: Azure Database for PostgreSQL Flexible Server handles minor version patches automatically and provides an in-portal upgrade path for major versions.

SSL/TLS certificates are managed automatically

HTTPS certificates on Azure App Service are issued and renewed by Microsoft without any manual intervention. There is no risk of a missed renewal causing the site to become inaccessible.

Encryption — data at rest and in transit

- *In transit:* All traffic between users and the application is encrypted via TLS 1.2/1.3, enforced by Azure App Service. Communication between the application and Azure Key Vault, and between the application and the Microsoft Graph mail service, also takes place over TLS.
- *At rest:* Azure App Service storage and Azure Database for PostgreSQL both encrypt data at rest using AES-256 by default, with no additional configuration required. For the SQLite pilot database stored on the App Service `/home` mount, the underlying Azure Files storage is also AES-256 encrypted at rest.

Secrets are stored securely in Azure Key Vault

The application requires several sensitive credentials — a JWT signing key, and the Azure client secret used for email. On a self-managed server these would be stored in a plain text configuration file on disk. Azure Key Vault stores them encrypted, with access logging and no file on disk. The App Service accesses Key Vault using a Managed Identity — no credential is needed to retrieve a credential, eliminating a class of secret-exposure risk entirely.

CERA is already in the Azure ecosystem

The application already uses CERA's Azure tenant for email (Microsoft Graph Mail.Send, with admin consent already granted). Hosting on Azure App Service keeps everything within the same administrative boundary — same tenant, same IT governance, same Microsoft relationship.

Data remains under CERA's control

The application and its database are hosted in CERA's own Azure subscription, in an EU datacentre (West Europe — Netherlands, or North Europe — Ireland). Microsoft has no access to the data. This is fully compliant with GDPR and consistent with NIS2 requirements applicable to regulatory authorities.

Compliance certifications are inherited

Azure holds ISO/IEC 27001, SOC 2 Type II, and GDPR-ready certifications. A self-managed server would require CERA to independently certify and maintain its own infrastructure security posture.

Reduced IT burden long-term

There is no server to maintain, no OS to patch, no hardware to replace, and no process manager to configure. If traffic grows or the application is extended in future, scaling is a configuration change rather than a hardware procurement.

> **EP (IT):** Encryption for data at rest or in transit?

Addressed above under the "Encryption" heading in Section 2.

---

3. Proposed architecture

The following Azure services are required:

| Service | Purpose | Estimated cost |

|---|---|---|

| Azure App Service — Basic B1 | Hosts and runs the Python web application | ~€13/month |

| Azure Key Vault | Secure storage for application secrets and credentials | ~€2/month |

| Azure Database for PostgreSQL Flexible Server | Production-grade database (pilot can defer this — see note) | ~€25/month |

| *(Azure App Registration)* | Already exists — used for email sending, no changes needed | No cost |

Estimated total: ~€40/month at full production.

For the initial pilot period, the PostgreSQL database can be deferred and the application can run with a built-in SQLite database at no additional cost, reducing the pilot cost to approximately **€15/month**.

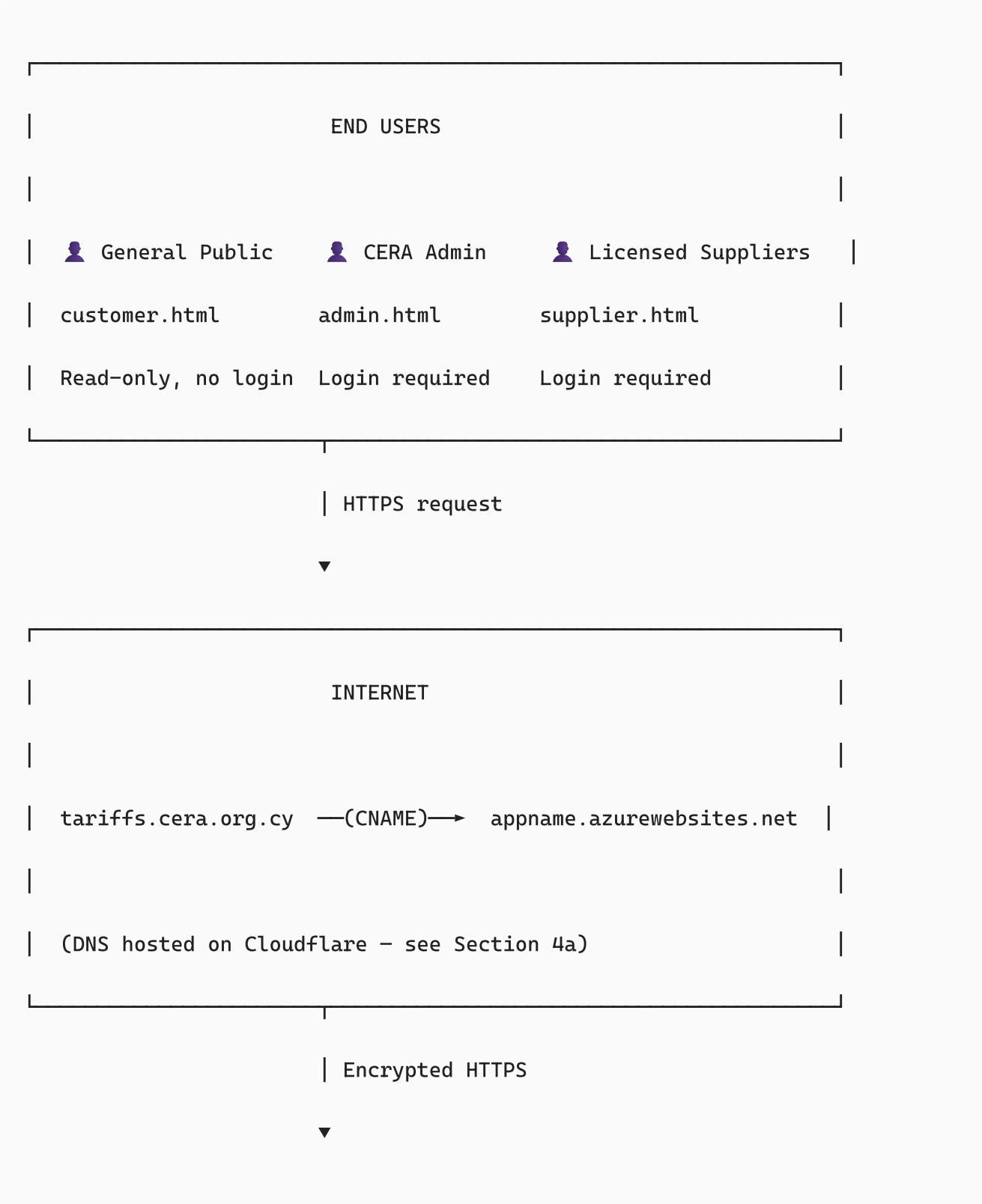
Regarding SQLite for the pilot: SQLite is appropriate for the pilot because user concurrency will be very low (a small number of CERA staff and a handful of suppliers). SQLite is stored

on the Azure `/home` persistent mount, which is AES-256 encrypted at rest. It is not exposed to the network — only the application itself can access the file. The security posture is equivalent to PostgreSQL for a single-server pilot scenario. The migration to PostgreSQL when concurrency becomes a concern is covered in Section 7.

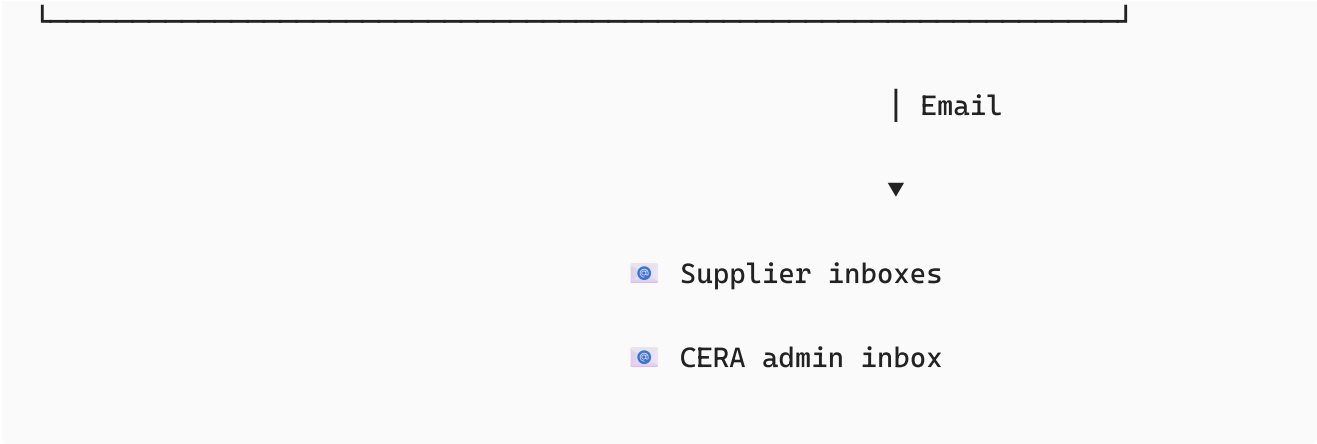
---

4. How the architecture works

Architecture diagram




MICROSOFT AZURE – CERA TENANT (EU Datacentre)
Azure App Service – Basic B1
SSL/TLS Certificate ← Managed & auto-renewed
Uvicorn + FastAPI (Python)
Business logic · REST API · JWT authentication
Auto-restart & always-on – built-in
STORAGE
SQLite (pilot
/home mount)
–or–
PostgreSQL
(production)
KEY VAULT
JWT key
Client secret
Tenant ID
Client ID
DB creds
AZURE ACTIVE DIR.
(CERA Tenant)
App Registration
Mail.Send permission
(already granted)
Azure Mail



Step-by-step request flow

1. A user opens a browser and navigates to `tariffs.cera.org.cy`
2. DNS resolves the domain to the Azure App Service via a CNAME record

> **EP (IT):** Domain hosting on Cloudflare — is it ok? How can it be deployed?

(see Section 4a for Cloudflare-specific steps)

3. Azure App Service handles the HTTPS connection using its automatically managed SSL certificate
4. The Python application processes the request — serving pages, handling logins, reading and writing tariff data
5. Secrets (credentials, signing keys) are read from Azure Key Vault — nothing sensitive is stored in configuration files
6. For email events (tariff submitted, approved, rejected, supplier invited) the application calls Azure Mail.Send using the existing CERA app registration
7. The encrypted response is returned to the user's browser

4a. DNS configuration when the domain is hosted on Cloudflare

If `cera.org.cy` DNS is managed via Cloudflare, the steps are slightly different from a standard DNS provider:

1. Log in to the Cloudflare dashboard and select the `cera.org.cy` zone
2. Add a **CNAME** record:

- **Name:** `tariffs`

- **Target:** `cera-tariff-tool.azurewebsites.net` (the Azure App Service default hostname)

- **Proxy status:** set to **DNS only** (grey cloud, not orange) — Azure manages its own SSL certificate; Cloudflare proxying would interfere with Azure's certificate issuance

3. In the Azure Portal, open the App Service → **Custom domains** → **Add custom domain**

4. Enter `tariffs.cera.org.cy` and follow the domain verification steps (Azure will ask for a TXT record to be added in Cloudflare to confirm ownership)
5. Once the custom domain is verified, enable the **Managed certificate** — Azure will issue and auto-renew the HTTPS certificate for `tariffs.cera.org.cy`

This is a standard process. Azure has a built-in guide that shows the exact DNS records needed at each step.

---

5. Three user roles, three pages

| Role | Page | Access |

|---|---|---|

| General public | customer.html | No login required. Read-only. Compares tariffs and estimates consumption. |

| CERA admin staff | admin.html | Login required. Manages suppliers, approves or rejects tariff submissions, manages regulated charges. |

> 💬 **EP (IT):** This page will be visible to the public?

> 📝 **PK:** See note below.

| Licensed suppliers | supplier.html | Login required. Submits and edits their own tariffs. Receives email notifications on approval or rejection. |

Regarding admin.html visibility: The admin page is publicly reachable at its URL (like any web page), but it displays only a login form. Without valid credentials it shows nothing — no data, no controls, no information about the system. This is standard practice for web admin panels. If CERA prefers to restrict the URL entirely (so that only CERA office IP addresses can reach admin.html), this can be done via Azure App Service access restrictions at no additional cost.

> 📝 **PK:** We should enable the IP restriction for admin.html before go-live.

User accounts are created by CERA admin staff through the admin panel. Suppliers receive an email invitation with a link to set their own password.

---

6. Security — questions answered

Password recovery process

A "Forgot password" link is available on both the admin and supplier login screens. Clicking it sends a time-limited reset link (valid 1 hour) to the user's registered email address. The link is single-use — it cannot be reused after the password has been changed. There is no ability to recover or view an existing password; only a reset is possible.

MFA / 2FA

The application implements two-factor authentication for all logins. After entering a correct email and password, the user receives a six-digit one-time code by email. The code expires after 10 minutes and is single-use. There is an option in the admin panel to temporarily disable OTP for specific accounts (e.g. during initial testing) — this can be re-enabled at any time.

> 📝 **PK:** I can enable/disable 2FA per user for testing purposes from the admin panel.

What if credentials are stolen / invalid data is submitted

- **Stolen credentials:** Without the OTP code delivered to the registered email address, stolen credentials alone are not sufficient to log in. Account lockout applies after 3 consecutive failed attempts (locked for 3 hours). The account holder is notified by email when a lockout occurs.
- **Invalid or malicious data submissions:** All inputs are validated server-side. Supplier tariff submissions are held in a pending state and must be reviewed and explicitly approved by a CERA admin before they become visible to the public. No supplier-submitted data is published automatically.

How user credentials are generated and what policy rules apply

- Accounts are created by CERA admin staff only — suppliers cannot self-register
- When an account is created, the supplier receives an email with a link to set their own password
- Passwords must be at least 8 characters (enforced server-side)
- Password policy for stronger requirements (length, complexity) can be tightened in the application configuration if CERA requires it

> 📝 **PK:** To be added prior to deployment.

How passwords are stored

Passwords are never stored in plain text. When a password is set or changed, it is processed through **bcrypt**, a one-way hashing algorithm specifically designed for password storage. bcrypt incorporates a random salt (preventing rainbow table attacks) and a work factor (making brute-force attacks computationally expensive). Even if the database file were obtained, passwords cannot be recovered from the stored hashes.

Can users change their passwords

Yes. Both admin and supplier users can change their own password from within the application after logging in (Profile / Account settings area). There is currently no enforced periodic password change. If CERA's security policy requires periodic rotation, a password expiry mechanism can be added.

> 📝 **PK:** To be added prior to deployment.

---

7. What IT needs to do

One-time setup tasks

1. **Create an Azure App Service** (Basic B1 tier) within CERA's existing Azure subscription, with Python 3.12 runtime selected
2. **Create an Azure Key Vault** and grant the App Service access to it using a Managed Identity (no credentials required — Azure handles this internally)
3. **Store the following secrets in Key Vault:** JWT signing key (a random 64-character string), Azure Tenant ID, Azure Client ID, Azure Client Secret, sender email address, admin notification email address, application base URL
4. **Generate a new Azure client secret** for the existing app registration (the one used for tariff-tool email sending) in Azure Portal → App registrations → Certificates & secrets. Set expiry to 12 months. Store the new secret in Key Vault. The previous secret used for local development should be deleted.
5. **Add a DNS CNAME record** pointing `tariffs.cera.org.cy` to the App Service hostname — follow the Cloudflare-specific steps in Section 4a above
6. **Add the custom domain** in App Service settings and enable the managed SSL certificate
7. **Deploy the application** by uploading a ZIP file of the project folder via the Azure Portal or Azure CLI (full instructions are in the companion AZURE_DEPLOYMENT_CHECKLIST.md)
8. **Enable Always On** in App Service configuration to prevent the app from sleeping between requests
9. **Create the first admin account** via the Azure SSH console (instructions in AZURE_DEPLOYMENT_CHECKLIST.md Section 8)

Ongoing (minimal)

- **Rotate the Azure client secret once per year** — a calendar reminder should be set. The process: generate a new secret in the App Registration, update the value in Key Vault, delete the old secret. The application picks up the new value automatically on next restart. Estimated time: 15 minutes.
- **Bug detection and fixing:** Application errors are logged in Azure App Service → Monitoring → Log stream. For the pilot period, the Regulatory Affairs team (who owns

the application) should be the first point of contact for reported issues. IT's involvement would be limited to infrastructure-level problems (App Service unavailability, certificate issues, Key Vault access problems). Application-level bugs are handled by the development team.
- **Review Azure Security Centre recommendations** periodically (monthly or quarterly)
- **Database backup:** For the SQLite pilot, a periodic manual download of the `tariff.db` file from the App Service SSH console is sufficient. For PostgreSQL production, Azure provides automated daily backups with point-in-time restore for up to 35 days — no action required.

Source code administration

The application source code is currently held on the CERA workstation where it was developed. For production, it is recommended that the source code be transferred to a CERA-controlled repository — either an Azure DevOps repository (already available within CERA's Microsoft tenant at no extra cost) or a private GitHub repository. This ensures that code changes are tracked, deployments are repeatable, and the application is not dependent on a single workstation. The Regulatory Affairs team and the development team should agree on ownership. IT can create the Azure DevOps repository if preferred.

---

8. Production upgrade path — SQLite to PostgreSQL

The pilot will run with SQLite. When the tool moves to full production — or when the number of active suppliers makes concurrent access a concern — the database should be upgraded to Azure Database for PostgreSQL Flexible Server.

Additional cost: ~€25/month on top of the App Service. Total production cost ~€40/month.

How the migration works (step by step)

1. **Create the Azure Database for PostgreSQL Flexible Server** in the Azure Portal (same resource group as the App Service)

- Choose the same region (West Europe)

- Select the Burstable B1ms tier (~€25/month) — sufficient for this workload

- Note the server hostname, database name, admin username, and password

2. **Store the database credentials in Key Vault** (hostname, database name, username, password as separate secrets)
3. **Export the existing SQLite data:**

```bash
```

```
# On the App Service SSH console
cd /home/site/wwwroot/tariff-api
python -c "
import sqlite3, json
conn = sqlite3.connect('/home/tariff.db')
conn.row_factory = sqlite3.Row
tables = conn.execute("SELECT name FROM sqlite_master WHERE type='table'").fetchall()
data = {}
for t in tables:
    data[t['name']] = [dict(r) for r in conn.execute(f'SELECT * FROM {t["name"]}')]
print(json.dumps(data))
" > /tmp/export.json
```

4. **Update the application connection string** — in the App Service environment variables, set:

```
DATABASE_URL = postgresql://username:password@hostname.postgres.database.azure.com/tariffdb?sslmode=require
```

5. **Import the data** into PostgreSQL using the development team's migration script (a one-time operation, estimated time: a few hours including testing)
6. **Restart the App Service** — the application detects the PostgreSQL connection string automatically and connects to the new database

The application code does not change. SQLAlchemy (the database layer used by the application) supports both SQLite and PostgreSQL with the same code — only the connection string differs.

---

9. Summary

| Requirement | How it is met |

|---|---|

| Public access over HTTPS | Azure App Service with managed SSL certificate |

| Secure credential storage | Azure Key Vault with Managed Identity access |

| Email notifications | Existing CERA Azure app registration (Mail.Send already granted) |

| Data sovereignty | CERA Azure tenant, EU datacentre |

| GDPR compliance | Azure EU hosting, inherited Microsoft certifications |

| Encryption at rest | AES-256 on Azure Files (SQLite) and PostgreSQL storage |

| Encryption in transit | TLS 1.2/1.3 enforced on all connections |

| Two-factor authentication | Email OTP on every login |

| Password security | bcrypt hashing — passwords never stored in plain text |

| Account protection | Lockout after 3 failed attempts, email notification |

| Automatic security patching | Managed by Microsoft on App Service platform |

| Python/runtime updates | One-click version change in Azure Portal |

| High availability (pilot) | App Service Basic B1 (99.95% SLA) |

| Cloudflare DNS compatibility | CNAME + DNS-only mode — full instructions in Section 4a |

| Source code administration | Recommended: Azure DevOps repository within CERA tenant |

| Cost | ~€15/month pilot · ~€40/month full production |

---

10. Contact

For questions about the application code, configuration values, or deployment steps, contact the Regulatory Affairs / CERA project lead.

Document version 2.0 — July 2026. Revised to address IT review comments on v1.0.